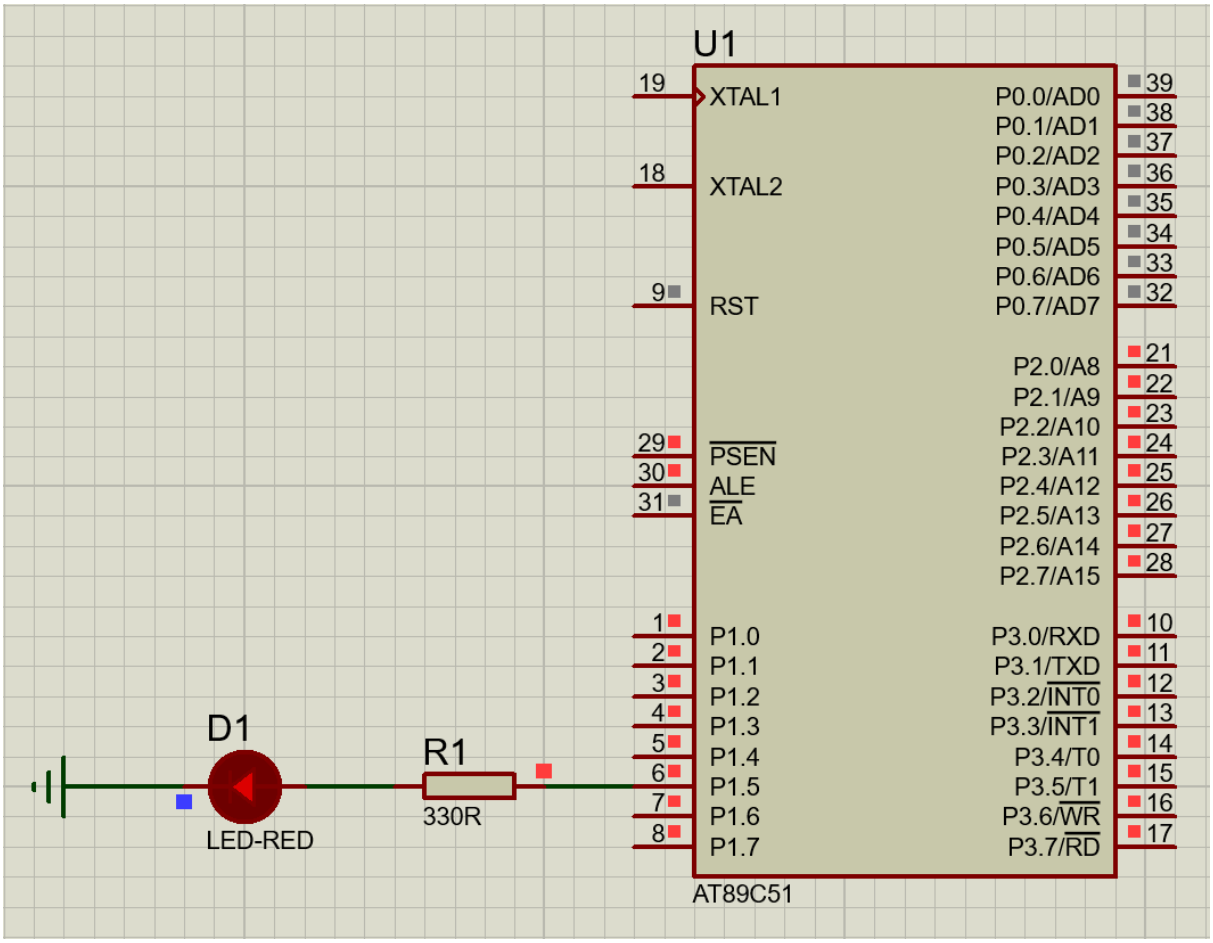
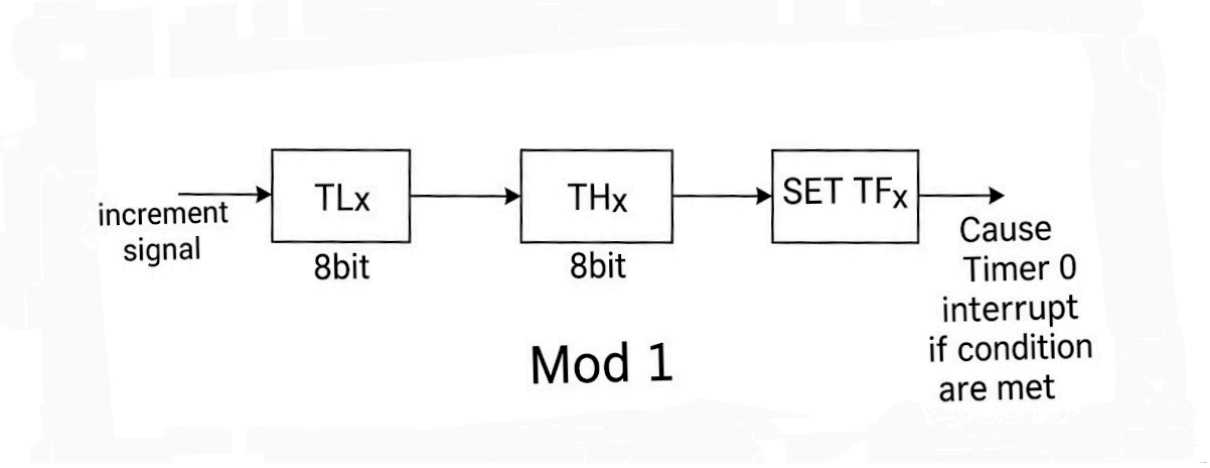
# 5️⃣ Timer 0, Mode 1 - Pin Toggle

**Aim**: To toggle bit P1.5 continuously using Timer 0 in Mode 1.




**Apparatus Required**:

- 8051 microcontroller
- Resistor
- Breadboard
- Connecting wires
- Power supply (5V)
- Keil uVision IDE

**Theory**:

- Timers in the 8051 microcontroller are special registers used to generate accurate delays and time intervals.
- The 8051 has 2 timers (Timer 0 & Timer 1), and each can operate in different modes.
- In this experiment, Timer 0 in Mode 1 is used.
- In this 16-bit mode, the timer uses 2 registers (`TH0` & `TL0`) to count from a loaded initial value up to its maximum (65535). After reaching the maximum, it overflows and sets the timer overflow flag (`TF0`).
- Using hardware timers is more efficient and accurate than software delay loops because timers operate independently of CPU instructions, ensuring precise timing.

**Calculations (For 50ms Delay)**:

- **Crystal Frequency:** $12MHz$
- **Machine Cycle:** $1\mu s$
- **Required Delay:** $50ms$
- **Counts Needed:** $\frac{50\text{ ms}}{1\ \mu\text{s}} = 50,000\text{ counts}$
- **Initial Value:** $65536 - 50000 = 15536$
- **Hexadecimal Conversion:** $15536 = 0\text{x}3\text{CB}0$
- **Register Values:** `TH0` $= 0\text{x}3\text{C}$, `TL0` $= 0\text{xB}0$

**Working**:

- Timer 0 in Mode 1 (16-bit timer) is used to generate a 50ms delay.
- An interrupt is used to increment a counter.
- After 20 interrupts ($20 \times 50ms = 1$ second), the target pin (P1.5) toggles its state.

**Program Code**:

```
#include <reg51.h>

sbit LED = P1^5; // Define LED at P1.5
unsigned char count = 0;

void timer0_ISR(void) interrupt 1 {
    // Reload timer values for next 50ms
    TH0 = 0x3C;
    TL0 = 0xB0;
    count++;

    // Toggle LED after 1 second (20 * 50ms = 1000ms = 1s)
    if (count >= 20) {
        LED = ~LED; // Toggle LED
        count = 0;  // Reset counter
    }
}

void main() {
    TMOD = 0x01; // Timer 0, Mode 1 (16-bit mode)

    TH0 = 0x3C;  // Load initial value (High byte)
    TL0 = 0xB0;  // Load initial value (Low byte)

    EA = 1;      // Enable Global Interrupts
    ET0 = 1;     // Enable Timer 0 Interrupt
    TR0 = 1;     // Start Timer 0

    while(1);    // Infinite loop, MCU waits for interrupts
}
```

**Result**: The LED at P1.5 was toggled continuously using the timer.

**Precautions**:

- Ensure the timer initial value is calculated correctly.
- Avoid overflow errors.